From a consumption point of view, we can say that metrics often are graphed (in dashboards) or are used as triggers for alerts in the context of monitoring and analytics. But metrics are also useful and, indeed, used as an input for programs. For example, a horizontal pod autoscaler (HPA) in Kubernetes may scale the number of replicas up or down, based on some custom metric, such as the number of requests handled by a service.

2.4.1 Instrumentation

Let's see how we can extend our `echo` service to emit metrics. Listing 2.7 shows the `echo` service instrumented with metrics. To be precise, we're using the Prometheus Go library for the instrumentation. Note that listing 2.7 only shows the relevant fragments, not the complete Go code, which is available via the code repo in code/ch02/metrics/main.go (http://mng.bz/wvWq).

Listing 2.7 Example Go program, instrumented to emit Prometheus metrics

```
func handleEcho(w http.ResponseWriter, r *http.Request) {
    message := r.URL.Query().Get("message")
    log.WithFields(log.Fields{
        "service": "echo",
    }).Info("Got input ", message)
    if rand.Intn(100) > 90 {
        log.WithFields(log.Fields{
            "service": "echo",
        }).Error("Something really bad happened :(")
       invokes.WithLabelValues(strconv.Itoa(http.StatusInternalServerError)).Inc()
        w.WriteHeader(500)
        return
    }
    invokes.WithLabelValues(strconv.Itoa(http.StatusOK)).Inc()
    fmt.Fprintf(w, message)
}
var (
    registry = prometheus.NewRegistry()
    invokes  = promauto.NewCounterVec(
        prometheus.CounterOpts{
            Name: "echo_total",
            Help: "Total invocations of the echo service endpoint.",
        },
        []string{"http_status_code"},
    )
)
func main() {
    rand.Seed(time.Now().UnixNano())
    registry.MustRegister(invokes)
    log.SetFormatter(&log.JSONFormatter{})
    http.HandleFunc("/echo", handleEcho)
    http.Handle("/metrics", promhttp.HandlerFor(
        registry,
        promhttp.HandlerOpts{
            EnableOpenMetrics: true,
```

Increments our metric with the context-specific label carrying the HTTP status code

Sets up the Prometheus registry

Defines our metric

Registers our metric

Defines the /metrics endpoint handler